SIMATS ENGINEERING COLLEGE

C PROGRAMMING — CO1 ASSIGNMENT 2

Design, Implement and Validate a Reliable Billing Program
for a Rural Electricity Co-operative

Tasks 1–4 | Total assignment: 50 marks | Individual, 1 week

Name	AKSHAYA R S
Register Number	1924260209
Course Code	CSA0209B
GitHub Repository Link	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING-1924260209

Contents

- Task 1 — Organise the Program with Header Files and Conditional Compilation (3 Marks)
- Task 2 — Read and Print Data Correctly (4 Marks)
- Task 3 — Control Type Conversion and Precision (4 Marks)
- Task 4 — Trace the Compilation Process (4 Marks)

Task 1 – Organise the Program with Header Files and Conditional Compilation

Bloom's level: K3 – Apply. The constants, function prototypes and a debugging switch are moved into the header file `billing.h`. The program `task01_billing.c` includes it and uses the same tariff constants for every bill.

1.1 Structure of a C program

Part	Where it is in this project	Purpose
Header files / preprocessor directives	`#include <stdio.h>` and `#include "billing.h"`	Bring in library declarations and the project's own constants and prototypes before compilation.
Global declarations	Constants (`#define`) and function prototypes in billing.h	Make names visible to every function; defined once, so a tariff change is made in one place.
main()	Reads the three inputs, calls the functions, prints the bill	Entry point: execution starts here and ends when main returns.
Other functions	read_uint, consumption, energy_charge, total_bill, print_rupees, print_bill	One job per function, so each can be tested and reused.

1.2 The header file billing.h

```
#ifndef BILLING_H
#define BILLING_H

#include <stdio.h>

/* ---- Tariff constants (all money in paise) ---- */
#define METER_MAX            65535U   /* 16-bit meter register maximum */
#define HOUSEHOLD_MAX        999999U  /* largest valid household number */
#define SLAB1_UNITS          100U
#define SLAB2_UNITS          200U
#define RATE1_PAISE          0LL      /* first 100 units   */
#define RATE2_PAISE          225LL    /* units 101 to 200  */
#define RATE3_PAISE          450LL    /* units above 200   */
#define FIXED_CHARGE_PAISE   5000LL   /* Rs 50.00 per bill */

/* ---- Debug support: active only when compiled with -DDEBUG ---- */
#ifdef DEBUG
#define DEBUG_PRINT(...) printf("[DEBUG] " __VA_ARGS__)
#else
#define DEBUG_PRINT(...) ((void)0)
#endif

/* ---- Function prototypes ---- */ int          read_uint(const char *prompt, unsigned
int max, unsigned int *out); unsigned int consumption(unsigned int previous, unsigned int
current); long long    energy_charge(unsigned int units); long long
total_bill(unsigned int units); void         print_rupees(long long paise); void
print_bill(unsigned int household, unsigned int units,                          long long
energy, long long total); #endif /* BILLING_H */
```

A header should contain declarations (constants, prototypes, types), not function bodies. The constants are in paise so that all money is integer arithmetic. `DEBUG_PRINT` is a macro that prints only when the program is compiled with `-DDEBUG`; otherwise it becomes `((void)0)`, a statement that does nothing.

1.3 Include guards

A header may be reached more than once in one translation unit, for example directly and again through another header. Declarations such as macros and function prototypes can be repeated harmlessly, but a definition of a `struct`, `enum` or typedef cannot. The guard `#ifndef BILLING_H / #define BILLING_H / ... / #endif` makes the preprocessor skip the body the second time. Demonstration with a header that has no guard (noguard.h) and one that has (guarded.h), each included twice:

```
struct tariff { unsigned int slab1; unsigned int slab2; };      /* noguard.h */
```

Compile without guard: `gcc -c g_noguard.c`

```
In file included from g_noguard.c:2: noguard.h:1:8: error:
redefinition of 'struct tariff'     1 | struct tariff { unsigned
int slab1; unsigned int slab2; };
      |        ^~~~~~ In file
included from g_noguard.c:1:
noguard.h:1:8: note: originally defined here     1 | struct
tariff { unsigned int slab1; unsigned int slab2; };
      |        ^~~~~~
```

Compile with guard: `gcc -c g_guard.c` — compiles with no error and no output.

1.4 #include <file> and #include "file"

#include <file> searches only the system include directories (and any added with `-I`); it is used for standard headers such as stdio.h. **#include "file"** searches first the directory of the file that contains the directive, and then the same places as the angle form; it is used for the project's own headers. Test: `#include <billing.h>` in ang.c:

Command: `gcc -c ang.c`

```
ang.c:1:10: fatal error: billing.h: No such file or directory
1 | #include <billing.h>      |        ^~~~~~~~~~~ compilation
terminated.
```

The header is in the current directory, not a system directory. Adding `-I.` to the command (`gcc -I. -c ang.c`) makes it compile, but the correct fix is `#include "billing.h"`.

1.5 Program and conditional compilation (#ifdef DEBUG)

```
#include "billing.h"

/* Reads one unsigned integer in the range 0..max.    Returns 1 on
success, 0 on invalid input. */ int read_uint(const char *prompt,
unsigned int max, unsigned int *out) {     unsigned int value;

    printf("%s", prompt);     if (scanf("%u", &value)
!= 1) {         printf("Error: not a number.\n");
return 0;     }     if (value > max) {
printf("Error: value must be 0 to %u.\n", max);
return 0;     }
    *out = value;     DEBUG_PRINT("read_uint:
accepted %u\n", value);     return 1; }

/* Units consumed, including a 16-bit meter rollover */ unsigned int
consumption(unsigned int previous, unsigned int current) {
unsigned int units = (current - previous) & METER_MAX;
```

```
    DEBUG_PRINT("consumption: previous=%u current=%u units=%u\n",
previous, current, units);     return units; }

/* Energy charge in paise from the three slabs */ long long
energy_charge(unsigned int units) {     long long charge;

    if (units <= SLAB1_UNITS)         charge = units * RATE1_PAISE;
else if (units <= SLAB2_UNITS)         charge = (units - SLAB1_UNITS) *
RATE2_PAISE;     else         charge = (SLAB2_UNITS - SLAB1_UNITS) *
RATE2_PAISE                + (units - SLAB2_UNITS) * RATE3_PAISE;

    DEBUG_PRINT("energy_charge: units=%u charge=%lld paise\n", units, charge);     return
charge; }

/* Total bill in paise = energy charge + fixed charge */ long long
total_bill(unsigned int units) {
    long long total = energy_charge(units) + FIXED_CHARGE_PAISE;

    DEBUG_PRINT("total_bill: total=%lld paise\n", total);     return
total; }

/* Prints an amount given in paise as rupees and paise, e.g. Rs 162.50 */ void
print_rupees(long long paise) {     printf("Rs %lld.%02lld", paise / 100, paise %
100); }

void print_bill(unsigned int household, unsigned int units,                long long energy,
long long total) {     printf("\n--- Electricity Bill ---\n");     printf("Household Number:
%u\n", household);     printf("Units Consumed: %u\n", units);     printf("Energy Charge: ");
print_rupees(energy);              printf("\n");     printf("Fixed Charge: ");
print_rupees(FIXED_CHARGE_PAISE);  printf("\n");     printf("Total Bill: ");
print_rupees(total);               printf("\n"); }

int main(void) {     unsigned int household, previous, current,
units;     long long energy, total;

    if (!read_uint("Enter household number: ", HOUSEHOLD_MAX, &household)) return 1;     if
(!read_uint("Enter previous reading: ", METER_MAX, &previous))      return 1;     if
(!read_uint("Enter current reading: ", METER_MAX, &current))        return 1;

    units   = consumption(previous, current);       energy =
energy_charge(units);          total    =  total_bill(units);
print_bill(household, units, energy, total);     return 0;
}
```

Compile: `gcc task01_billing.c -Wall -Wextra -std=c11 -o t1_rel` and with debugging: `gcc -DDEBUG task01_billing.c -Wall -Wextra -std=c11 -o t1_dbg` — both compile with no warnings.

Normal build (inputs 101, 1200, 1530): 330 units = 100 × Rs 2.25 + 130 × Rs 4.50 = Rs 225.00 + Rs 585.00 = Rs 810.00; plus Rs 50.00 fixed = Rs 860.00.

```
Enter household number: 101
Enter previous reading: 1200
Enter current reading: 1530

--- Electricity Bill ---
Household Number: 101
Units Consumed: 330
Energy Charge: Rs 810.00
Fixed Charge: Rs 50.00
Total Bill: Rs 860.00
```

Debug build (same inputs, compiled with -DDEBUG) — the intermediate values appear:

```
Enter household number: 101
[DEBUG] read_uint: accepted 101
Enter previous reading: 1200
[DEBUG] read_uint: accepted 1200
Enter current reading: 1530
[DEBUG] read_uint: accepted 1530
[DEBUG] consumption: previous=1200 current=1530 units=330
[DEBUG] energy_charge: units=330 charge=81000 paise
[DEBUG] energy_charge: units=330 charge=81000 paise
[DEBUG] total_bill: total=86000 paise

--- Electricity Bill ---
Household Number: 101
Units Consumed: 330
Energy Charge: Rs 810.00
Fixed Charge: Rs 50.00
Total Bill: Rs 860.00
```

Rollover test (previous 65400, current 120): (65536 − 65400) + 120 = 256 units; 100 × Rs 2.25 + 56 × Rs 4.50 = Rs 477.00; total Rs 527.00.

```
Enter household number: 102
Enter previous reading: 65400
Enter current reading: 120

--- Electricity Bill ---
Household Number: 102
Units Consumed: 256
Energy Charge: Rs 477.00
Fixed Charge: Rs 50.00
Total Bill: Rs 527.00
```

1.6 Alternative: a run-time debug variable

Instead of `#ifdef DEBUG`, the trace statement is guarded by a variable that is set while the program runs (here, from a command-line argument):

```
static int debug = 0;                       /* set in main: debug = (argc > 1); */
#define TRACE(...) do { if (debug) printf(__VA_ARGS__); } while (0)
```

The same function `energy_charge()` was timed in bench_debug.c (100,000,000 calls, best of three runs) in three forms: **release** (TRACE removed by the preprocessor), **run-time flag** (debug off) and **-DDEBUG** (always on, so only its size is compared). Compiled with `-Wall -Wextra -std=c11`, at -O0 and -O2

Version	-O0 time	-O2 time	Code size, -O2 (text bytes)	Debug text in executable?
#ifdef, no -DDEBUG (release)	0.419 s (4.2 ns/call)	0.104 s (1.0 ns/call)	1692	No (0 matches)
Version	**-O0 time**	**-O2 time**	**Code size, -O2 (text bytes)**	**Debug text in executable?**
Run-time flag, debug off	0.439 s (4.4 ns/call)	0.149 s (1.5 ns/call)	1820	Yes (format string and call kept)
#ifdef with -DDEBUG	not timed (prints 100 million lines)	not timed	1780	Yes

Turning the run-time flag on prints the trace without recompiling:

```
energy_charge: units=0 charge=0
energy_charge: units=1 charge=0
energy_charge: units=2 charge=0
```

Time complexity. A call costs O(1) in both versions; for n bills the total is O(n) in both. The run-time version executes one extra load, compare and branch in every call, which measured about 0.2 ns per call at -O0 (within the noise of the runs) and about 0.45 ns per call at -O2. For a district of 800,000 bills that is under 0.4 ms, so the cost is negligible here, but it is a cost the #ifdef release build never pays. **Space complexity.** The #ifdef release build contains no debug code. The run-time build keeps the format string and the call in the executable: 128 more bytes of code for this one trace point, growing as O(k) for k trace points. **Comparison.** #ifdef gives the smallest and fastest release program but needs a recompilation to switch debugging on; the run-time flag can be switched on in the field (for example to investigate a disputed bill) at a small, constant cost in time and space.

Evidence for Task 1: header with include guards (1.2, 1.3); working DEBUG mode (1.5); program structure explained (1.1); alternative analysed with measurements (1.6).

Task 2 – Read and Print Data Correctly

Bloom's level: K3 – Apply. Input and output use `scanf` and `printf` with the correct format specifier for each type, and invalid input is detected.

2.1 Format specifiers

C type	printf	scanf	Used in this program for
int	%d	%d	signed whole numbers
unsigned int	%u	%u	household number, meter readings, units
long	%ld	%ld	values that need at least 32 bits (size is platform-dependent)
long long	%lld	%lld	money in paise, district totals
float	%f	%f	(not used for money)
double	%f or %lf	%lf	fractional measurements; scanf needs %lf
char	%c	%c	a single character
char array (string)	%s	%s	text; scanf needs room for the characters plus '\0'

```
#include <stdio.h>

int main(void) {     int i = -42;
unsigned int u = 4000000000U;     long
l = 1234567890123L;     long long ll =
9000000000LL;     float f = 2.5f;
double d = 3.14159265358979;     char
c = 'A';     char s[] = "Jyothi";
long long paise = 16250, small = 5;

    printf("int           %%d   : %d\n", i);
printf("unsigned int  %%u   : %u\n", u);
printf("long          %%ld  : %ld\n", l);
printf("long long     %%lld : %lld\n", ll);
printf("float         %%f   : %f\n", f);
printf("double        %%lf  : %lf\n", d);
printf("char          %%c   : %c\n", c);
printf("string        %%s   : %s\n", s);

    printf("\nRupees and paise from integer paise:\n");     printf("%lld
paise -> Rs %lld.%02lld\n", paise, paise / 100, paise % 100);
printf("%lld paise -> Rs %lld.%02lld\n", small, small / 100, small % 100);
printf("%lld paise -> Rs %lld.%lld   (wrong: %%02lld missing)\n",
small, small / 100, small % 100);     return 0;
}
```

Compile: `gcc task02_specifiers.c -Wall -Wextra -std=c11 -o t2_spec` — *no warnings. Output:*

```
int           %d   : -42 unsigned
int    %u      :  4000000000  long
%ld    :  1234567890123  long  long
%lld : 9000000000 float         %f
: 2.500000 double          %lf   :
3.141593 char            %c    : A
string        %s   : Jyothi

Rupees and paise from integer paise:
16250 paise -> Rs 162.50
5 paise -> Rs 0.05
5 paise -> Rs 0.5   (wrong: %02lld
missing)
```

2.2 Printing rupees and paise from an integer amount

Money is held as an integer number of paise. The rupees are `paise / 100` and the paise part is `paise % 100`. The paise part must be printed with `%02lld` so that 5 paise appears as .05 and not .5; the last two lines of the output above show the difference. 16250 paise is printed as Rs 162.50. The code assumes a non-negative amount, which the program guarantees because the tariff has no negative charges. **2.3 What happens with a wrong specifier**

```
#include <stdio.h>

int main(void) {     long long big = 5000000000LL;     /*
5,000,000,000 paise */     long      lng = 4000000000L;
int       neg = -1;     long      x   = -1L;     double    d
= 2.5;

    printf("1) %%d  with long long 5000000000 : %d\n", big);
printf("2) %%d  with long      4000000000 : %d\n", lng);
printf("3) %%u  with int -1               : %u\n", neg);
printf("4) %%d  with double 2.5           : %d\n", d);

    /* scanf with the wrong specifier: %d writes only 4 bytes of an 8-byte long */
if (sscanf("7", "%d", &x) == 1)
        printf("5) sscanf(\"7\", \"%%d\", &x) with long x = -1 -> x = %ld\n", x);
return 0;
}
```

Compile: `gcc task02_wrongspec.c -Wall -Wextra -std=c11 -o t2_wrong`. *gcc itself warns about every mismatch (-Wformat):*

```
 task02_wrongspec.c:11:50: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long lo
task02_wrongspec.c:12:50: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long in
task02_wrongspec.c:14:50: warning: format '%d' expects argument of type 'int', but argument 2 has type 'double'
```

Output (the behaviour is undefined; these values were observed with gcc 13.3.0 on 64-bit Linux):

```
1) %d  with long long 5000000000 : 705032704
2) %d  with long      4000000000 : -294967296
3) %u  with int -1               : 4294967295
4) %d  with double 2.5           : -1539291712
5) sscanf("7", "%d", &x) with long x = -1 -> x = -4294967289
```

- Test 1 and 2: `%d` reads only 32 bits, so 5,000,000,000 (which needs 33 bits) is printed as 705,032,704 and 4,000,000,000 as −294,967,296. A district total printed this way would be wrong with no error message.
- Test 3: `%u` prints the bit pattern of −1 as an unsigned number, 4,294,967,295.
- Test 4: `%d` given a double prints an unrelated number because a double is passed in a different register; the value can change from run to run.
- Test 5: `%d` in scanf stores only 4 bytes into an 8-byte long. The upper 4 bytes of x (all ones from −1) are left unchanged, so reading 7 gives −4,294,967,289 instead of 7.

2.4 Checking the return value of scanf

`scanf` returns the number of items it converted, or EOF when the input ends. If a letter is typed where a number is expected it returns 0 and leaves the letter in the input, so the program must (1) test the return value, and (2) discard the rest of the line, otherwise the next `scanf` fails on the same character forever. The values must also be range checked: 70000 is a valid unsigned int but not a valid 16-bit reading. The function `read_uint` in task02_io.c does all three; only this function differs from the Task 1 program, the rest of the file is unchanged.

```
/* Reads one unsigned integer in the range 0..max.
   Keeps asking until the input is valid. Returns 1 on success,    0
if the input ends (EOF). */ int read_uint(const char *prompt,
unsigned int max, unsigned int *out) {     unsigned int value;
int status, c;

    for (;;) {
        printf("%s", prompt);         status = scanf("%u", &value);
/* number of items converted */         if (status == EOF) {
printf("\nError: end of input.\n");             return 0;         }
if (status == 1) {             if (value <= max) {                 *out =
value;                 DEBUG_PRINT("read_uint: accepted %u\n", value);
return 1;             }             printf("Error: value must be 0 to %u. Try
again.\n", max);         } else {             printf("Error: not a number.
Try again.\n");         }         /* discard the rest of the bad line so
scanf can start afresh */         while ((c = getchar()) != '\n' && c != EOF)
;
    }
}
```

Compile: `gcc task02_io.c -Wall -Wextra -std=c11 -o t2_io` *— no warnings. Test with invalid inputs: letters (abc), a reading above 65535 (70000), a negative number (−5), then valid values 1200 and 1530:*

```
Enter household number: 101
Enter previous reading: abc
Error: not a number. Try again.
Enter previous reading: 70000 Error: value
must be 0 to 65535. Try again.
Enter previous reading: -5 Error: value
must be 0 to 65535. Try again.
Enter previous reading: 1200
Enter current reading: 1530

--- Electricity Bill ---
Household Number: 101
Units Consumed: 330
Energy Charge: Rs 810.00
Fixed Charge: Rs 50.00
Total Bill: Rs 860.00
```

The negative number is read by `%u` as 4,294,967,291, which is above the limit, so it is rejected by the range check. Each bad value was rejected and the program asked again; the final bill is correct (330 units, Rs 860.00).

2.5 Alternative: fgets() with strtoul() or sscanf()

The whole line is read into a buffer with `fgets` and then converted with `strtoul`. The conversion reports exactly where it stopped, so trailing characters can be detected.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>
#include "billing.h"

/* Reads a whole line with fgets, then converts it with strtoul. */
int read_uint(const char *prompt, unsigned int max, unsigned int
*out) {     char buf[32];     char *end;     unsigned long value;
const char *p;     int c;

    for (;;) {         printf("%s", prompt);         if (fgets(buf, sizeof buf,
stdin) == NULL) {             printf("\nError: end of input.\n");             return
0;         }         if (strchr(buf, '\n') == NULL && !feof(stdin)) {   /* line too
long */             while ((c = getchar()) != '\n' && c != EOF)                 ;
printf("Error: line too long. Try again.\n");             continue;         }
p = buf;         while (isspace((unsigned char)*p))             p++;         if
(!isdigit((unsigned char)*p)) {             printf("Error: not a number. Try
again.\n");             continue;         }         errno = 0;         value =
strtoul(p, &end, 10);         while (isspace((unsigned char)*end))
end++;         if (*end != '\0') {                     /* trailing garbage such as
12abc */             printf("Error: not a number. Try again.\n");
continue;         }         if (errno == ERANGE || value > max) {
printf("Error: value must be 0 to %u. Try again.\n", max);             continue;
}         *out = (unsigned int)value;         return 1;     } }

int main(void) {     unsigned int household,
previous, current, units;

    if (!read_uint("Enter household number: ", HOUSEHOLD_MAX, &household)) return 1;
if (!read_uint("Enter previous reading: ", METER_MAX, &previous))      return 1;
if (!read_uint("Enter current reading: ", METER_MAX, &current))        return 1;

    units = (current - previous) & METER_MAX;
    printf("\nHousehold %u: previous %u, current %u -> %u
units\n",            household, previous, current, units);
return 0; }
```

Compile: `gcc task02_fgets.c -Wall -Wextra -std=c11 -o t2_fg` *— no warnings. Same invalid inputs as above:*

```
Enter household number: 101
Enter previous reading: abc
Error: not a number. Try again.
Enter previous reading: 70000 Error: value
must be 0 to 65535. Try again.
Enter previous reading: -5
Error: not a number. Try again.
Enter previous reading: 1200
Enter current reading: 1530

Household 101: previous 1200, current 1530 -> 330 units
```

Recovery from partly valid input. The line `12abc` is typed as the previous reading, then 1530 and 1700 (the typed text is not echoed because the input comes from a file):

scanf version:

```
Enter household number: Enter previous reading: Enter current reading: Error: not a number. Try again.
Enter current reading:
--- Electricity Bill ---
Household Number: 101
Units Consumed: 1518
Energy Charge: Rs 6156.00
Fixed Charge: Rs 50.00
Total Bill: Rs 6206.00
```

fgets version:

```
Enter household number: Enter previous reading: Error: not a number. Try again.
Enter previous reading: Enter current reading:
Household 101: previous 1530, current 1700 -> 170 units
```

The scanf version accepted “12” as the previous reading, then failed on “abc” at the next prompt, so the bill was calculated for 1518 units (Rs 6,206.00), a silent wrong bill. The fgets version rejected the whole line, so the final reading pair (1530, 1700) gave the correct 170 units. The fgets version also handles a line longer than the buffer:

```
Enter household number: 101
Enter previous reading: 12345678901234567890123456789012345678901234567890123456789012345 Error: line too long. Try again.
Enter previous reading: 1200
Enter current reading: 1530

Household 101: previous 1200, current 1530 -> 330 units
```

Measurements (bench_input.c, gcc `-Wall -Wextra -std=c11`, no optimisation):

```
Parsing one number from a string (5000000 repetitions)
L = 1 digits: sscanf   64.2 ns/call   strtoul   17.8 ns/call
L = 5 digits: sscanf   80.9 ns/call   strtoul   25.7 ns/call
L = 9 digits: sscanf  107.6 ns/call   strtoul   32.0 ns/call
Reading 2000000 lines from a file: fscanf   56.0 ns/line   fgets+strtoul   26.6 ns/line (2000000, 2000000 read)
```

Time complexity. Converting a line of L characters is O(L) in both approaches, because each character is examined a constant number of times. The measurements show the constant: `sscanf` rose from 64 ns (1 digit) to 108 ns (9 digits), `strtoul` from 18 ns to 32 ns, and reading a file of 2,000,000 lines took 56 ns per line with fscanf and 27 ns per line with fgets + strtoul. **Space.** `scanf` stores the value directly in the variable (4 bytes) and needs no program buffer; the fgets approach needs a buffer of B bytes (32 here), O(1) extra space chosen in advance, and B must be larger than the longest valid line. Both use the library’s internal stdio buffer. **Recovery.** After a failure scanf leaves the bad characters in the input, so the program must discard them; it can also accept the valid beginning of a line. fgets removes the complete line each time, so the next prompt always starts on a clean line and the whole line can be validated; only a line longer than the buffer needs extra code.

Task 3 – Control Type Conversion and Precision

Bloom's level: K4 – Analyse.

3.1 Places where different types are combined

The table lists each expression in the billing program (task01_billing.c) where operands of different types meet, and what the compiler does.

Expression	Operand types	Conversion performed
`(current - previous) & METER_MAX`	unsigned int, unsigned int, unsigned int	None needed. The subtraction wraps modulo 2^32 and the mask keeps the low 16 bits, giving the rollover result.
`units * RATE1_PAISE`	unsigned int × long long	Usual arithmetic conversions: units is converted to long long.
`(units - SLAB1_UNITS) * RATE2_PAISE`	unsigned int − unsigned int, then × long long	The subtraction is done as unsigned (safe, because units > 100 here); the result is then converted to long long.
`(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE`	unsigned int constants × long long	Evaluated as unsigned 100U, then converted to long long.
`energy_charge(units) + FIXED_CHARGE_PAISE`	long long + long long	None.
`paise / 100`	long long ÷ int constant	100 is converted to long long; integer division discards the remainder.
`value <= max`	unsigned int, unsigned int	None. If one operand were an int, the int would be converted to unsigned.
`printf("%lld", total)`	long long passed to a varargs function	No conversion by the compiler: the specifier must match the type exactly.

3.2 Surprising results

```c
#include <stdio.h>

int main(void) {     unsigned int units = 199;     long long paise = 16250;     long
long district_total = 4000000000LL;      /* 800,000 x Rs 50.00 in paise */
unsigned int previous = 65400, current = 120;     int balance = -1;     unsigned int
limit = 1;     unsigned short a = 65535, b = 1;     double x = 0.1, y = 0.2;

    /* 1. Integer division */     printf("1) 199 / 100            = %u   (not 1.99)\n",
units / 100);     printf("   16250 paise / 100    = %lld   (the 50 paise are lost)\n",
paise / 100);     printf("   16250 paise / 100.0  = %.2f\n", paise / 100.0);

    /* 2. Explicit cast of a large long long to int */     printf("2) (int)4000000000
= %d   (not 4000000000)\n", (int)district_total);

    /* 3. Unsigned subtraction wraps */     printf("3) 120u - 65400u        = %u
(not -65280)\n", current - previous);     printf("   (120u - 65400u) & 65535 = %u
(the correct rollover result)\n",            (current - previous) & 65535U);

    /* 4. Signed / unsigned comparison */     printf("4) -1 < 1u
: %s   (-1 is converted to %u)\n",            (balance < limit) ? "true"
: "false", (unsigned int)balance);



    /* 5. Integer promotion */     printf("5) (unsigned short)65535 + 1 = %d   (promoted to int, no
wrap)\n", a + b);     printf("   (unsigned short)(65535 + 1) = %d   (cast back to 16 bits)\n",
(unsigned short)(a + b));

    /* 6. Floating point */     printf("6) 0.1 + 0.2 == 0.3     :
%s   (0.1 + 0.2 = %.17f)\n",            (x + y == 0.3) ? "true" :
"false", x + y);     return 0;
}
```

Compile: `gcc task03_conversions.c -Wall -Wextra -std=c11 -o t3_conv`. *One warning is produced, and it is deliberate: gcc reports the signed/unsigned comparison used in surprise 4.*

```
task03_conversions.c: In function 'main': task03_conversions.c:29:21: warning: comparison of integer
expressions of different signedness: 'int' and 'unsi
```

Output:

```
1) 199 / 100            = 1   (not 1.99)
   16250 paise / 100    = 162   (the 50 paise are lost)
   16250 paise / 100.0  = 162.50
2) (int)4000000000      = -294967296   (not 4000000000)
3) 120u - 65400u        = 4294902016   (not -65280)
   (120u - 65400u) & 65535 = 256   (the correct rollover result)
4) -1 < 1u              : false   (-1 is converted to 4294967295)
5) (unsigned short)65535 + 1 = 65536   (promoted to int, no wrap)
   (unsigned short)(65535 + 1) = 0   (cast back to 16 bits)
6) 0.1 + 0.2 == 0.3     : false   (0.1 + 0.2 = 0.30000000000000004)
```

- **1. Integer division.** Both operands are integers, so the division is done in integers and the fraction is discarded: 199 / 100 is 1, and 16250 paise / 100 gives 162 rupees, losing 50 paise. The correct method is to use `/ 100` for the rupees and `% 100` for the paise, or to divide by a floating-point 100.0 for display only.
- **2. Explicit cast of a large value.** 4,000,000,000 paise (the fixed charges of 800,000 households) does not fit in a 32-bit int (maximum 2,147,483,647). The cast keeps the low 32 bits and the result is −294,967,296, a negative district total. gcc does not warn about an explicit cast.

- **3. Unsigned subtraction.** 120u − 65400u cannot be negative; it wraps to 2^32 − 65280 = 4,294,902,016. Masking with 65535 keeps the low 16 bits and gives the correct 256 units.
- **4. Signed/unsigned comparison.** In `-1 < 1u` the int −1 is converted to unsigned int (4,294,967,295), so the comparison is false. gcc warns with -Wsign-compare (part of -Wextra).
- **5. Integer promotion.** The unsigned short operands 65535 and 1 are promoted to int before the addition, so the sum is 65536 and does not wrap. Only the cast back to unsigned short gives the 16-bit result 0.
- **6. Floating point.** 0.1 and 0.2 have no exact binary representation, so their sum is 0.30000000000000004 and the equality test with 0.3 is false.

3.3 Rounding rule

The tariff itself produces whole paise (rates are 0, 225 and 450 paise per unit), so no rounding is needed. A fractional charge appears as soon as a percentage is applied, for example a 5% surcharge. **Rule used: round half up, once, in integer arithmetic, on the final charge of each bill:** `(charge * percent + 50) / 100`. Truncation would always favour the customer; rounding in floating point could differ in the last paise.

```
#include <stdio.h>

/* Rounding rule: round half up, once, in integer arithmetic.
   surcharge = charge * percent / 100, rounded to the nearest paise. */
static long long surcharge(long long charge_paise, long long percent) {
return (charge_paise * percent + 50) / 100; }

int main(void)
```

```
{     long long charges[] = { 16250, 16270, 27501, 101
};     int n = (int)(sizeof charges / sizeof
charges[0]);

    printf("5%% surcharge on a charge in paise\n");     printf("%8s %12s %10s
%10s\n", "charge", "exact", "truncated", "rounded");     for (int i = 0; i <
n; i++) {         long long c = charges[i];         printf("%8lld %12.2f
%10lld %10lld\n",                c, c * 5 / 100.0, c * 5 / 100, surcharge(c,
5));     }     return 0;
}
```

Compile: `gcc task03_rounding.c -Wall -Wextra -std=c11 -o t3_round` *— no warnings. Output:*

```
5% surcharge on a charge in paise   charge        exact  truncated    rounded    16250       812.50        812
813
   16270       813.50        813        814
   27501      1375.05       1375       1375
     101         5.05          5          5
```

For 16250 and 16270 the exact value is a half (812.50 and 813.50), and the rule rounds up to 813 and 814, while truncation gives 812 and 813. For 27501 and 101 the fraction is below a half and both methods agree.

3.4 Alternative: float arithmetic, rounded once at the end

Every bill is Rs 12.34 (1234 paise). The total of n bills is added in float rupees (and in double for comparison), converted to paise and rounded once at the end, and compared with the exact total from integer paise.

```
#include <stdio.h>
#include <time.h>

int main(void) {     const long long ns[] = { 1000LL, 10000LL, 100000LL,
1000000LL, 10000000LL };     printf("%10s %16s %18s %14s %14s\n", "n bills",
"exact (Rs)",             "float sum (Rs)", "float err", "double err");     for
(int k = 0; k < 5; k++) {         float fsum = 0.0f;         double dsum = 0.0;
        long long ptotal = 0;                 /* exact total in paise */
for (long long i = 0; i < ns[k]; i++) {             fsum += 12.34f;
/* every bill is Rs 12.34 */             dsum += 12.34;
ptotal += 1234;         }
        long long fpaise = (long long)((double)fsum * 100.0 + 0.5);   /* rounded once at the end
*/         long long dpaise = (long long)(dsum * 100.0 + 0.5);         printf("%10lld
%10lld.%02lld %12lld.%02lld %11lld p %11lld p\n",                ns[k], ptotal / 100, ptotal %
100,                fpaise / 100, fpaise % 100, fpaise - ptotal, dpaise - ptotal);     }

    /* cost per value */     const long long N =
100000000LL;     float fs = 0.0f;     long long ps
= 0;     clock_t t0 = clock();     for (long long
i = 0; i < N; i++) fs += 12.34f;     clock_t t1 =
clock();     for (long long i = 0; i < N; i++) ps
+= 1234;     clock_t t2 = clock();
    printf("\n%lld additions: float %.3f s, integer paise %.3f s  (checks: %.0f, %lld)\n",
N, (double)(t1 - t0) / CLOCKS_PER_SEC, (double)(t2 - t1) / CLOCKS_PER_SEC, (double)fs, ps);
printf("Storage per value: float %zu bytes, long long %zu bytes\n",            sizeof(float),
sizeof(long long));     return 0;
}
```

Compile: `gcc task03_float.c -Wall -Wextra -std=c11 -o t3_float` *— no warnings. Output:*

```
   n bills       exact (Rs)     float sum (Rs)      float err     double err
1000      12340.00        12339.91          -9 p           0 p
     10000     123400.00       123416.82        1682 p           0 p
    100000    1234000.00      1233490.63      -50937 p           0 p
   1000000   12340000.00     12270234.00    -6976600 p           0 p
  10000000  123400000.00    137990688.00  1459068800 p           3 p

100000000 additions: float 0.284 s, integer paise 0.240 s  (checks: 268435456, 123400000000)
Storage per value: float 4 bytes, long long 8 bytes
```

Error growth. The integer-paise total has zero error at every n. The float total is already 9 paise wrong for 1,000 bills, Rs 16.82 wrong for 10,000, Rs 509.37 for 100,000, Rs 69,766 for 1,000,000 and Rs 1.46 crore

too large for 10,000,000 bills. The error is far larger than linear growth would give because a float has only 24 bits of precision: once the running total is large, the spacing between floats (8 at 123 million, 32 at 268 million) is of the same order as one bill, so each addition is rounded badly. In the 100-million-addition run the float total stopped growing at 268,435,456 (2^28), where adding 12.34 changes nothing, although the exact total is 1,234,000,000. In general, the worst-case error of n floating-point additions grows in proportion to n times the machine epsilon times the size of the total, i.e. O(n) or worse here. Double (53 bits) stayed exact up to 1,000,000 bills but was 3 paise wrong at 10,000,000, so it delays the error but does not remove it.

Time and space. Both methods do n additions, so the time is O(n) and the extra space is O(1). Per value the measured costs are about 2.8 ns for a float addition and 2.4 ns for an integer-paise addition (0.284 s and 0.240 s for 100,000,000 additions, no optimisation), so the integer method is not slower. A float uses 4 bytes per stored value against 8 bytes for long long, but the 4 bytes cannot hold the amounts needed: the integer method pays 4 extra bytes per value for exact results.

Evidence for Task 3: conversion points identified (3.1), six demonstrated surprises with explanations (3.2), a consistent rounding rule (3.3), float-versus-integer measurements (3.4).

Task 4 – Trace the Compilation Process (4 Marks)

Bloom's level: K3 – Apply. The program of Task 1 (task01_billing.c with billing.h) is taken through every stage, one command at a time. **4.1 The stages and their output**

Stage	Command	Output file	What it contains
Preprocessing	`gcc -E task01_billing.c -o task01_billing.i`	task01_billing.i	C text with macros expanded, headers pasted in and comments removed.
Compilation	`gcc -S task01_billing.c -o task01_billing.s`	task01_billing.s	x86-64 assembly language (text).
Assembly	`gcc -c task01_billing.c -o task01_billing.o`	task01_billing.o	Machine code in a relocatable ELF object; calls to library functions are still unresolved.
Linking	`gcc task01_billing.o -o task01_billing`	task01_billing	Executable: object code joined with the C library.

Size of each file produced:

```
file            bytes   lines
task01_billing.c     2799     90
billing.h            1242     32
task01_billing.i    23819    916
task01_billing.s     6984    411
task01_billing.o     4336
task01_billing      16368      -
```

The 90-line program becomes 916 lines after preprocessing because stdio.h and the types it needs are pasted in. The `file` command identifies the last two stages:

```
task01_billing.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
task01_billing:   ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib
```

The first bytes of the object file (7f 45 4c 46 are "ELF"):

```
000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  >.ELF............<
000010
```

4.2 What the preprocessor removed or replaced

- **Comments:** 5 lines with /* ... */ in the source, 0 in the preprocessed file.
- **#define lines:** all 11 in billing.h disappear, and every use is replaced by its value.
- **#include:** stdio.h and billing.h are replaced by their contents.
- **Conditional code:** every DEBUG_PRINT(...) becomes `((void)0)`. Compiling with -DDEBUG instead keeps the four [DEBUG] strings; without it the preprocessed file has none.

The function energy_charge in the preprocessed file shows the constants replaced (SLAB1_UNITS by 100U, RATE2_PAISE by 225LL, SLAB2_UNITS − SLAB1_UNITS by 200U − 100U):

```
long long energy_charge(unsigned int units); long long
total_bill(unsigned int units); void print_rupees(long
long paise); void print_bill(unsigned int household,
unsigned int units,                          long long
energy, long long total); # 2 "task01_billing.c" 2



int read_uint(const char *prompt, unsigned int max, unsigned int
*out) {     unsigned int value;

    printf("%s", prompt);     if (scanf("%u", &value)
!= 1) {         printf("Error: not a number.\n");
return 0;     }     if (value > max) {
printf("Error: value must be 0 to %u.\n", max);
return 0;     }
    *out =
value;
((void)0);
return 1; }
long long energy_charge(unsigned int
units) {     long long charge;

    if (units <= 100U)         charge =
units * 0LL;     else if (units <=
200U)         charge = (units - 100U) *
225LL;     else         charge = (200U
- 100U) * 225LL                + (units
- 200U) * 450LL;

    ((void)0);
return charge;
}
```

4.3 The assembly and object stages

The assembly for `consumption()` *(task01_billing.s). The expression* `(current - previous) & METER_MAX` *is the instructions subl and andl $65535:*

```
consumption:
.LFB1:
■.cfi_startproc
■endbr64
■pushq■%rbp
■.cfi_def_cfa_offset 16
■.cfi_offset 6, -16
■movq■%rsp, %rbp
■.cfi_def_cfa_register 6
■movl■%edi, -20(%rbp)
■movl■%esi, -24(%rbp)
■movl■-24(%rbp), %eax
■subl■-20(%rbp), %eax
■andl■$65535, %eax
■movl■%eax, -4(%rbp) ■movl■-
4(%rbp), %eax
■popq■%rbp
■.cfi_def_cfa 7, 8
■ret
■.cfi_endproc
.LFE1:
```

```
■.size■consumption, .-consumption
```

Symbols in the object file (`nm task01_billing.o`): T = defined here, U = undefined, to be supplied by the linker from the C library:

```
                 U __isoc99_scanf
                 U __stack_chk_fail
00000000000000c3 T consumption
00000000000000e4 T energy_charge
00000000000002df T main
0000000000000205 T print_bill
0000000000000167 T print_rupees
                 U printf
                 U putchar
                 U puts
0000000000000000 T read_uint
000000000000013e T total_bill
```

4.4 Compile-time, link-time and run-time errors

Compile-time error — the compiler rejects the source. A semicolon was removed from `units = consumption(previous, current);` (copy named e_compile.c):

```
gcc -Wall -Wextra -std=c11 -c e_compile.c
```

```
e_compile.c: In function 'main':
e_compile.c:85:44: error: expected ';' before 'energy'    85 |     units  = consumption(previous, current)
      |                                            ^
      |                                            ;
   86 |     energy = energy_charge(units);
      |     ~~~~~~
```

Link-time error — every file compiles but a function has no definition. When the multi-file version of Task 4.6 was linked without tariff.c, the calls in main.c could not be resolved:

```
gcc -Wall -Wextra -std=c11 input.c report.c main.c -o broken
```

```
/usr/bin/ld: /tmp/cciR5405.o: in function `main':
main.c:(.text+0x9b): undefined reference to `consumption' /usr/bin/ld: main.c:(.text+0xa8):
undefined reference to `energy_charge' /usr/bin/ld: main.c:(.text+0xb6): undefined
reference to `total_bill' collect2: error: ld returned 1 exit status
```

Run-time error — the program builds, then fails while running. A proposed “average units per household” (e_runtime.c) divides by the number of households, which is zero when nothing is read:

```
#include <stdio.h>

int main(int argc, char **argv) {     unsigned int total_units = 48000;     unsigned int
households = (unsigned int)(argc - 1);   /* 0 when no argument is given */

    (void)argv;     printf("Average units per household = %u\n",
total_units / households);     return 0;
}
```

It compiles with no warning, because the divisor is not known at compile time. Running it with no argument:

```
Floating point exception
rc=136
```

Exit status 136 means the program was stopped by signal 8 (SIGFPE). The fix is to test `households > 0` before dividing. The three kinds are reported by different tools (compiler, linker, operating system) at different times.

4.5 Warnings from -Wall -Wextra

A draft of the bill function (task04_warn.c) was compiled with the warning options. It built, but gcc reported four problems (shown here with the types the draft used):

```
#include <stdio.h>
#define SLAB1_UNITS 100U

long long bill(int units, int
category) {     long long energy = 0;
int spare = 0;

    if (units > SLAB1_UNITS)         energy =
(units - SLAB1_UNITS) * 225LL;     return
energy + 5000; }

int main(void) {     int units =
150;     long long total =
bill(units, 1);

    printf("Total paise: %d\n", total);
return 0;
}
```

```
task04_warn.c: In function 'bill': task04_warn.c:9:15: warning: comparison of integer expressions
signedness: 'int' and 'unsigned int     9 |     if (units > SLAB1_UNITS)      |             ^ task04
warning: unused variable 'spare' [-Wunused-variable]      7 |      int spare = 0;         |
task04_warn.c:4:31: warning: unused parameter 'category' [-Wunused-parameter]     4 | long long bil
int category)       |                           ~~~~^~~~~~~~ task04_warn.c: In function 'main':
task04_warn.c:19:27: warning: format '%d' expects argument of type 'int', but argument 2 has type
in    19 |     printf("Total paise: %d\n", total);
      |                          ~^     ~~~~~
      |                           |     |
      |                           int   long long int
      |                          %lld
```

Warning	Flag	What it told me	How it was resolved

comparison of integer expressions of different signedness	-Wsign-compare (-Wextra)	int units was compared with the unsigned constant 100U; a negative value would be converted to a huge unsigned number.	units declared unsigned int, as in Task 2.
unused variable 'spare'	-Wunused-variable (-Wall)	Leftover variable that does nothing.	Deleted.
unused parameter 'category'	-Wunused-parameter (-Wextra)	The category is not used by the single tariff.	Parameter removed (it can be added back when a category tariff exists).
format '%d' expects int, argument has type long long	-Wformat (-Wall)	The money value would be printed from only 32 bits (see Task 2.3).	Specifier changed to %lld.

The corrected version (task04_warn_fixed.c) compiles with no warnings and prints the right amount:

```
Total paise: 16250
```

4.6 Alternative: several source files with a shared header

The program is split into input.c (read_uint), tariff.c (consumption, energy_charge, total_bill), report.c (print_rupees, print_bill) and main.c, all including billing.h. The function bodies are the same as in the single file. Each .c file is compiled to an object file and the objects are linked. The Makefile records the dependencies:

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 OBJS =

input.o tariff.o report.o main.o

billing: $(OBJS)

■$(CC) $(OBJS) -o billing

%.o: %.c billing.h
■$(CC) $(CFLAGS) -c $< -o $@

clean:

■rm -f $(OBJS) billing
```

First build, then after editing only tariff.c (`touch tariff.c; make`), then after changing the header (`touch billing.h; make`):

```
gcc -Wall -Wextra -std=c11 -c input.c -o input.o
gcc -Wall -Wextra -std=c11 -c tariff.c -o tariff.o
gcc -Wall -Wextra -std=c11 -c report.c -o report.o
gcc -Wall -Wextra -std=c11 -c main.c -o main.o gcc
input.o tariff.o report.o main.o -o billing
```

```
gcc -Wall -Wextra -std=c11 -c tariff.c -o tariff.o
gcc input.o tariff.o report.o main.o -o billing
```

```
gcc -Wall -Wextra -std=c11 -c input.c -o input.o
gcc -Wall -Wextra -std=c11 -c tariff.c -o tariff.o
gcc -Wall -Wextra -std=c11 -c report.c -o report.o
gcc -Wall -Wextra -std=c11 -c main.c -o main.o gcc
input.o tariff.o report.o main.o -o billing
```

Only tariff.c was recompiled after the first edit, but changing billing.h forced all four files to be rebuilt, because every file includes it. The split program gives byte-for-byte the same output as the single file for the same input (checked with cmp), and -DDEBUG works in every file.

Build time (timing.py: mean of repeated builds with -Wall -Wextra -std=c11):

```
Real project (4 source files, about 90 lines), mean of 30
builds single file, full build (compile + link)      : 42.2 ms
four files, full build (4 compiles + link)    : 90.6 ms four
files, after editing tariff.c (1 + link) : 34.2 ms

Synthetic project: 4,000 functions (about 32001 source lines), mean of 3 builds
one file, full build                          : 3648 ms ten files, full build
: 3413 ms ten files, after editing one file (1 + link)   : 376 ms object files:
ten-file build 1197840 bytes in total; one-file build 1186632 bytes
```

Time complexity. Compiling a file costs about a fixed start-up time plus a term proportional to its size S. With one file every change costs O(S_total). With separate files a change to one file costs O(S_file) for the compile plus the link, which depends on the number of objects and symbols, not on the size of the code. For this 90-line program the start-up time dominates: a full four-file build (90.6 ms) is slower than the single file (42.2 ms), and an edit to one file (34.2 ms) is only a little faster. In the synthetic project of about 32,000 lines the full builds are about the same (3.6 s and 3.4 s), but an edit to one file takes 0.38 s instead of 3.6 s, about ten times less. A change to the shared header still costs a full rebuild. **Space.** The object files are extra files on disk: 8,384 bytes in total for the four small objects against 4,336 bytes for the single object, because each object file carries its own headers and symbol table (the machine code itself is 1,685 against 1,517 bytes). For the large project the totals were almost equal (1,197,840 against 1,186,632 bytes, about 1% more). The executables are the same size apart from a small difference.

Evidence for Task 4: output of every stage (4.1–4.3), one correctly classified example each of a compile-time, link-time and run-time error (4.4), warnings explained and resolved (4.5), the separate compilation alternative measured (4.6).

Design Challenge – Smart Meters and a Time-of-Day Tariff

The co-operative is moving to smart meters that report every 15 minutes, with a time-of-day tariff. The original design is re-evaluated part by part.

Effect on the data

- **Volume:** 4 readings per hour × 24 × 30 days = 2,880 readings per household per month, against 1 per month before. That is 11.5 million readings per month for a village (4,000 houses), 172.8 million for a taluk (60,000) and 2,304 million for a district (800,000). Even at 4 bytes per reading the district needs about 9.2 GB of readings each month, so the program must process a stream of readings instead of holding all of them in memory.
- **Billing rule:** the first-100-units slab is based on the whole month's consumption, while the time-of-day rate depends on the 15-minute interval, so both must be combined.

Decision for each part of the design

Part of the design	Decision	Reason
Money as integer paise, rounded once with a stated rule	**Retain**	Still the only way to keep bills exact to the paisa. Time-of-day rates can be held as paise per kWh and energy in whole Wh; the products are exact integers (units of 1/1000 paise) that are added and rounded once per bill.
Constants and tariff in billing.h	**Modify**	One slab table becomes a table of rate periods (start time, end time, rate). It is still defined in one place, but as data (a const array), not as single #defines.
Meter register and rollover logic	**Modify**	Each interval is a difference of two cumulative readings, so rollover handling must run for every interval, and the register width must be checked for the new meters.
Types	**Modify**	Interval energy fits in unsigned int; running sums per household and district totals stay long long.
Input with scanf, one household at a time	**Redesign**	2,880 values per household cannot be typed or read with interactive scanf. Read a file or stream line by line (fgets-style, with validation of every line) and process each interval as it arrives.
Single source file	**Redesign**	Split into input, tariff and report modules with a shared header (Task 4.6), so the tariff can change without recompiling input code, and each module can be tested alone.

Conclusion: the core ideas of the original design are **retained** (integer paise, central constants, explicit rollover handling, validation, one rounding rule). The tariff and data types are **modified**. The input stage and program structure are **redesigned**, because the number of readings grows by about 2,900 times and the tariff now depends on the time of each reading.

Execution and GitHub Record

Repository: **CSA02-<1924260209>-Assignment1**, one folder per task, each containing the source code, input files and output screenshots. Commit each task separately.

Folder	Files	Compile command(s)
Task01	billing.h, task01_billing.c, bench_debug.c, noguard.h, guarded.h, g_noguard.c, g_guard.c, ang.c	`gcc task01_billing.c -Wall -Wextra -std=c11 -o task01_billing` `gcc -DDEBUG task01_billing.c -Wall -Wextra -std=c11 -o task01_debug gcc -O2 bench_debug.c -o bench_debug`
Folder	**Files**	**Compile command(s)**
Task02	billing.h, task02_specifiers.c, task02_wrongspec.c, task02_io.c, task02_fgets.c, bench_input.c	`gcc task02_io.c -Wall -Wextra -std=c11 -o task02_io gcc task02_fgets.c -Wall -Wextra -std=c11` `-o task02_fgets`
Task03	task03_conversions.c, task03_rounding.c, task03_float.c	`gcc task03_float.c -Wall -Wextra -std=c11 -o task03_float`
Task04	task01_billing.c, task04_warn.c, task04_warn_fixed.c, e_runtime.c, split/ (input.c, tariff.c, report.c, main.c, billing.h, Makefile)	`gcc -E / -S / -c / link` as in 4.1 `make` in split/

Compiler used: `gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0` (from `gcc --version`). Every program was compiled with `-Wall -Wextra -std=c11`; the only warnings are the deliberate ones shown in Task 2.3, Task 3.2 and Task 4.5. All outputs in this report were produced by running the programs.

README.md must contain: name, register number, compiler and version, the exact compile command of each program, the steps to run each program, and a short note of the result of each task (for example: “Task 1: header with guards and DEBUG mode working; 330 units gives Rs 860.00”).

Suggested commits: “Task01: header file, include guards, DEBUG mode”; “Task02: input validation and format specifiers”; “Task03: conversion tests and rounding rule”; “Task04: compilation stages and split build”; “Add README and report”. Paste the repository link on the cover page and share the repository with the faculty.